

РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ

Факультет физико-математических и естественных наук

Кафедра прикладной информатики и теории вероятностей

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ № 4

дисциплина: Архитектура компьютера _

Студент: Ромеро Гаспар Фернандо Алексей

Группа: НКАбд-02-25

МОСКВА

2025 г.

Оглавление

Цель заботы

Теоретическое введение

2.1 Основные принципы работы компьютера

2.2 Ассемблер и язык ассемблера

2.3. Процесс создания и обработки программы на языке ассемблера

Выполнения лабораторной работы

Выполнение самостоятельной работы

Выводы

Список литературы

Цель работы

Освоение процедуры компиляции и сборки программ, написанных на ассемблере NASM.

Теоретическое введение

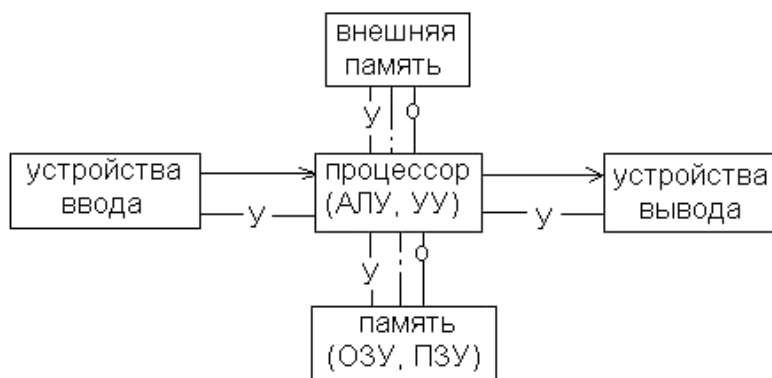
2. 1 Основные принципы работы компьютера

Современный компьютер представляет собой сложную систему, фундамент которой был заложен в середине XX века. Несмотря на колоссальный прогресс в скорости и миниатюризации, базовые принципы функционирования остаются неизменными. В основе лежат универсальная архитектурная модель фон Неймана, двоичная система счисления и принципы логической организации данных и команд. Понимание этих основ позволяет осмыслить, как аппаратное и программное обеспечение взаимодействуют для решения задач.

Архитектурная парадигма: модель фон Неймана

Разработанная в 1945 году Джоном фон Нейманом и его коллегами, эта модель определила структуру большинства последующих компьютеров. Её ключевые постулаты:

Принцип хранимой программы: Команды (инструкции программы) и данные хранятся в одной и той же памяти в двоичном виде. Это радикально отличало новую архитектуру от более ранних, где программа задавалась аппаратной коммутацией. Благодаря этому принципу компьютер стал универсальным: для решения новой задачи достаточно загрузить в память новую программу, а не перестраивать машину.



Последовательное выполнение команд: Команды выполняются процессором последовательно, одна за другой, если только сама программа не содержит инструкции перехода.

Структура из пяти основных блоков:

- Арифметико-логическое устройство (АЛУ): Выполняет математические операции и логические сравнения.
- Устройство управления (УУ): Координирует работу всех узлов компьютера, извлекает команды из памяти, декодирует их и инициирует выполнение. Вместе с АЛУ образует Центральный процессор (ЦПУ или CPU).
- Запоминающее устройство (память): Хранит программы и данные. Иерархия памяти (кэш, ОЗУ, ПЗУ, диск) позволяет балансировать между скоростью, объёмом и стоимостью.
- Устройства ввода: Обеспечивают поступление информации и команд (клавиатура, мышь, сенсоры).
- Устройства вывода: Представляют результаты работы (монитор, принтер).

Логический и физический уровни: двоичность и логические схемы

Принцип фон Неймана описывает логическую организацию. Физическая реализация опирается на следующие основы:

Двоичная система счисления: Из-за простоты технической реализации (состояние «есть сигнал» / «нет сигнала», «включено» / «выключено») вся информация кодируется комбинациями двух цифр: 0 и 1. Бит — минимальная единица информации. Последовательности битов образуют байты, числа, символы (кодировки ASCII, Unicode), команды процессора.

10-чная система	2-чная система	16-тиричная система
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F

Булева алгебра и логические вентили: Работу с битами описывает математическая логика (булева алгебра). Её физическими «кирпичиками» являются логические вентили — базовые электронные схемы (И, ИЛИ, НЕ, И-НЕ, ИЛИ-НЕ), реализующие простейшие

логические функции. Миллиарды таких вентилях, интегрированных в кристаллы, образуют процессоры, память и контроллеры. Из вентилях собираются более сложные схемы: триггеры (элементарные ячейки памяти, хранящие 1 бит), сумматоры, регистры, дешифраторы.

Название элемента (операции)	Российский стандарт	Зарубежный стандарт
Инвертор (НЕ)		
Конъюнктор (И)		
Дизъюнктор (ИЛИ)		
Штрих Шеффера (И-НЕ)		
Стрелка Пирса (ИЛИ-НЕ)		

Цикл выполнения команды: сердце работы процессора

Работа ЦПУ — это непрерывное повторение цикла команды (instruction cycle):

Выборка (Fetch): Устройство управления считывает следующую команду из оперативной памяти по адресу, указанному в счётчике команд (Program Counter), и помещает её в регистр команд.

Декодирование (Decode): Схемы процессора анализируют команду, определяя, какую операцию (сложение, сравнение, запись) и над какими данными выполнять.

Исполнение (Execute): АЛУ выполняет требуемую операцию. Это может быть арифметическое действие, обращение к памяти для чтения/записи данных или изменение счётчика команд для выполнения перехода.

Запись результата (Writeback): Результат операции записывается в регистр процессора или в память.

Современные процессоры используют методы конвейеризации (pipelining), когда разные стадии обработки нескольких команд выполняются одновременно, что резко повышает производительность.

Роль операционной системы: абстракция и управление

«Голое» аппаратное обеспечение (hardware) слишком сложно для прямого использования программами. Операционная система (ОС) выступает посредником, предоставляя критически

Абстракция аппаратных ресурсов: ОС скрывает сложность железа, предоставляя приложениям удобные интерфейсы (API) для работы с файлами, сетью, памятью.

Организация данных: Файловая система ОС структурирует информацию на дисках, обеспечивая её долговременное хранение, поиск и защиту.

Язык ассемблера — низкоуровневый язык программирования, представляющий собой символическую (мнемоническую) запись машинного кода. Каждая команда ассемблера соответствует одной инструкции процессора, обеспечивая прямой контроль над аппаратным обеспечением.

- Мнемоники: Команды процессора обозначаются мнемоническими кодами (например, MOV — переместить, ADD — сложить).
- Прямой доступ: Программист напрямую оперирует регистрами процессора и адресами памяти.
- Архитектурная зависимость: Язык специфичен для архитектуры процессора (x86, ARM и др.).

[illegible]

Ассемблер — программа-транслятор, преобразующая исходный код на языке ассемблера в исполняемый машинный код. Процесс преобразования называется ассемблированием.

Основные функции ассемблера:

- Трансляция мнемоник в опкоды: Преобразование символических команд в двоичные операционные коды (opcodes).
- Обработка меток: Замена символических имён (меток) на фактические адреса памяти.
- Обработка директив: Выполнение указаний для самого ассемблера (определение данных, резервирование памяти и т.д.).
- Генерация объектного файла: Создание объектного модуля (.obj, .o) для последующей компоновки.



Схема преобразования:

Исходный файл .asm → Ассемблер → Объектный файл .obj → Компоновщик → Исполняемый файл .exe

2.3. Процесс создания и обработки программы на языке ассемблера

1. Написание исходного кода (.asm): Программист создает текстовый файл с мнемоническими инструкциями (такими как MOV, ADD), директивами ассемблера (такими как SECTION, DB) и метками. Этот файл читаем для человека, но не исполняем процессором.

2. Ассемблирование (Assembling): Ассемблер (такой как NASM, MASM или GAS) обрабатывает файл .asm в два основных прохода:

2.1. Первый проход: Строит таблицу символов с адресами всех меток.

2.2. Второй проход: Генерирует машинный код, заменяя каждую мнемонику на её двоичный опкод и разрешая ссылки на метки с помощью таблицы символов.

Результат: Объектный файл (.obj или .o), содержащий машинный код, но с неразрешёнными внешними ссылками.

3. Компоновка (Связывание, Linking): Компоновщик (линкер) объединяет один или несколько объектных файлов с необходимыми библиотеками:

3.1. Разрешение внешних символов: Связывает вызовы функций библиотек (таких как printf или системные вызовы).

3.2. Назначение финальных адресов: Организует разделы кода и данных в единое адресное пространство.

3.3. Генерация исполняемого файла: Создаёт файл в специфическом формате операционной системы (PE для Windows, ELF для Linux).

4. Загрузка и выполнение: Загрузчик операционной системы (loader):

4.1. Загружает исполняемый файл в оперативную память (RAM).

4.2. Назначает реальные физические адреса (в системах с виртуальной памятью — виртуальные адреса).

4.3. Переходит на адрес точки входа программы.

4.4. CPU последовательно исполняет инструкции машинного кода.



Выполнения лабораторной работы

ВПрограмма Hello world!

Традиционно первая программа выводит на экран приветственное сообщение «Hello world!».

Мы создали каталог для работы с программами на языке ассемблера NASM:

```
fernando@ubuntu25-10:~$ mkdir -p ~/work/arch-pc/lab04
fernando@ubuntu25-10:~$
```

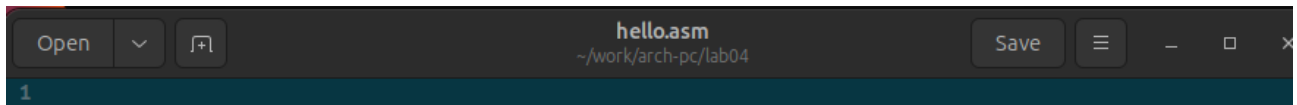
Перейдем в созданный каталог:

```
fernando@ubuntu25-10:~$ cd ~/work/arch-pc/lab04
fernando@ubuntu25-10:~/work/arch-pc/lab04$
```

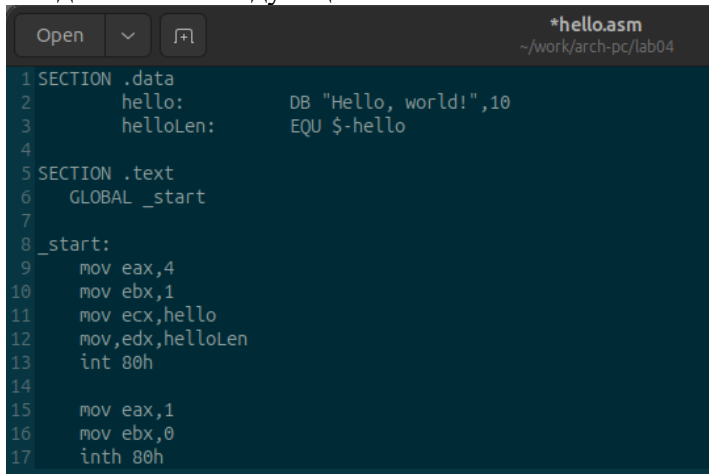
Создадим текстовый файл с именем hello.asm:

```
fernando@ubuntu25-10:~/work/arch-pc/lab04$ touch hello.asm
fernando@ubuntu25-10:~/work/arch-pc/lab04$ ls hello.asm
hello.asm
```

Откроем этот файл с помощью текстового редактора gedit:



Введем в него следующий текст:



Транслятор NASM

Преобразуем текст программы из файла hello.asm в объектный код:

```
fernando@ubuntu25-10:~/work/arch-pc/lab04$ nasm -f elf hello.asm
```

Объектный код записался в файл hello.o. Проверим это с помощью команды ls:

```
fernando@ubuntu25-10:~/work/arch-pc/lab04$ ls hello.asm hello.o
hello.asm  hello.o
```

Таким образом, имена всех файлов получаются из имени входного файла и расширения по умолчанию. NASM всегда создаёт выходные файлы в текущем каталоге.

Расширенный синтаксис командной строки NASM

Выполните следующую команду:

```
fernando@ubuntu25-10:~/work/arch-pc/lab04$ nasm -o obj.o -f elf -g -l list.lst hello.asm
```

Команда компилирует исходный файл hello.o в obj.o и создает список.

Это проверяется с помощью команды `ls`:

```
fernando@ubuntu25-10:~/work/arch-pc/lab04$ ls
hello.asm hello.o list.lst obj.o
```

Формат выходного файла elf. В него включены символы для отладки.

Компоновщик LD

Передадим объектный файл на обработку компоновщику:

```
fernando@ubuntu25-10:~/work/arch-pc/lab04$ ld -m elf_i386 hello.o -o hello
```

Проверьте с помощью команды `ls`, был ли создан файл `hello`:

```
fernando@ubuntu25-10:~/work/arch-pc/lab04$ ls
hello hello.asm hello.o list.lst obj.o
```

Компоновщик ld не предполагает по умолчанию расширений для файлов, но принято использовать следующие расширения:

- o – для объектных файлов;
- без расширения – для исполняемых файлов;
- tar – для файлов схемы программы;
- lib – для библиотек.

Ключ -o с последующим значением задаёт в данном случае имя создаваемого исполняемого файла. Выполним следующую команду:

Выполните следующую команду:

```
fernando@ubuntu25-10:~/work/arch-pc/lab04$ ld -m elf_i386 obj.o -o main
```

Исполняемый файл будет иметь имя «main». Объектный файл, из которого собран этот исполняемый файл имеет имя «obj.o».

Запуск исполняемого файла

Запустите созданный исполняемый файл, расположенный в текущем каталоге:

```
fernando@ubuntu25-10:~/work/arch-pc/lab04$ ./hello
Hello, world!
```

Выполнение самостоятельной работы

Задание 1

1. В каталоге ~/work/arch-pc/lab04 с помощью команды cp создайте копию файла hello.asm с именем lab4.asm

```
fernando@ubuntu25-10:~/work/arch-pc/lab04$ cp hello.asm lab4.asm
fernando@ubuntu25-10:~/work/arch-pc/lab04$ ls
hello  hello.asm  hello.o  lab4.asm  list.lst  main  obj.o
```

Задание 2

2. С помощью любого текстового редактора внесите изменения в текст программы в файле lab4.asm так, чтобы вместо Hello world! на экран выводилась строка с вашими фамилией и именем.

```
fernando@ubuntu25-10:~/work/arch-pc/lab04$ gedit lab4.asm

lab4.asm
~/work/arch-pc/lab04

1 SECTION .data
2     hello:      DB "Fernando Alexei Romero Gaspar",10
3     helloLen:   EQU $-hello
4
5 SECTION .text
6     GLOBAL _start
7
8 _start:
9     mov eax,4
10    mov ebx,1
11    mov ecx,hello
12    mov edx,helloLen
13    int 80h
14
```

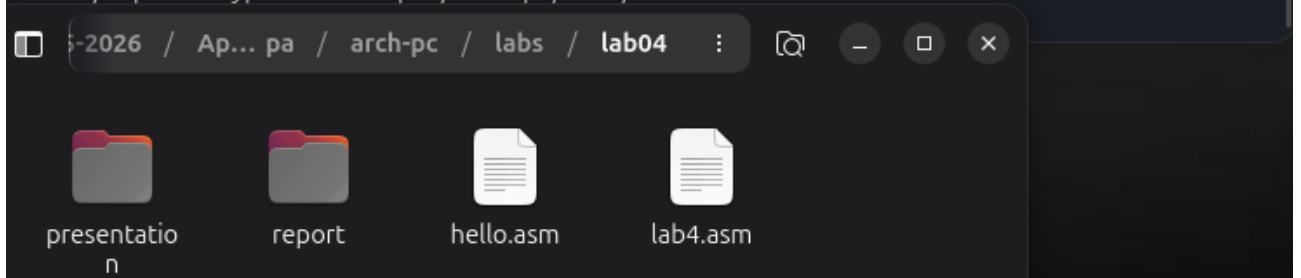
Задание 3

3. Оттранслируйте полученный текст программы lab4.asm в объектный файл. Выполните компоновку объектного файла и запустите получившийся исполняемый файл

```
fernando@ubuntu25-10:~/work/arch-pc/lab04$ nasm -f elf lab4.asm
fernando@ubuntu25-10:~/work/arch-pc/lab04$ ld -m elf_i386 lab4.o -o lab4
fernando@ubuntu25-10:~/work/arch-pc/lab04$ ./lab4
Fernando Alexei Romero Gaspar
```

4. Скопируйте файлы hello.asm и lab4.asm в Ваш локальный репозиторий в каталог ~/work/study/2023-2024/"Архитектура компьютера"/arch-pc/labs/lab04/. Загрузите файлы на Github.

```
fernando@ubuntu25-10:~/work/arch-pc/lab04$ cp ~/work/arch-pc/lab04/hello.asm ~/work/study/2023-2026/"Архитектура компьютера"/arch-pc/labs/lab04
fernando@ubuntu25-10:~/work/arch-pc/lab04$ cp ~/work/arch-pc/lab04/lab4.asm ~/work/study/2023-2026/"Архитектура компьютера"/arch-pc/labs/lab04
```



```
fernando@ubuntu25-10:~/work/study/2025-2026/Архитектура компьютера/arch-pc/labs/lab04$ git add .
fernando@ubuntu25-10:~/work/study/2025-2026/Архитектура компьютера/arch-pc/labs/lab04$ git commit -m 'feat
(labs): add hello. asm and lab4.asm'
[master 4e6307c] feat(labs): add hello. asm and lab4.asm
 2 files changed, 36 insertions(+)
 create mode 100644 labs/lab04/hello.asm
 create mode 100644 labs/lab04/lab4.asm
```

```
fernando@ubuntu25-10:~/work/study/2025-2026/Архитектура компьютера/arch-pc/labs/lab04$ git push
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 4 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (6/6), 681 bytes | 681.00 KiB/s, done.
Total 6 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), completed with 2 local objects.
To github.com:AlexeiFRG1/study_2025-2026_arch-pc.git
 8b46185..4e6307c  master -> master
```

Выводы

В ходе лабораторных работ они освоили основные процедуры компиляции и ассемблирования программ, написанных на ассемблере NASM. Они также изучили и применили команды для преобразования текстового кода программы в объектный код, передачи объектного кода компоновщику и выполнения программы. Кроме того, они получили базовое понимание основных принципов работы компьютеров, ассемблера, языка ассемблера, а также процесса создания и запуска программ на ассемблере.

Список литературы

1. Таненбаум Э., Остин Т. «Архитектура компьютера». 6-е изд. – СПб.: Питер, 2013. – Классический учебник, подробно рассматривающий все уровни организации, от логических вентилях до архитектур процессоров.
2. Институт инженеров по электротехнике и электронике (IEEE), Computer Society. «Introduction to Computer Architecture» <https://www.computer.org/education/introduction-to-computer-architecture>
3. Хабрахабр. Разделы «Железо» и «Процессоры». – <https://habr.com/ru/hub/processors/> –
4. Национальный исследовательский университет «Высшая школа экономики» (НИУ ВШЭ). Факультет компьютерных наук. Открытые курсы и материалы. – <https://cs.hse.ru/>
5. Khan Academy на русском. Курс «Основы компьютерных наук». <https://ru.khanacademy.org/computing/ap-computer-science-principles>
6. Калашников О.А. «Ассемблер — это просто. Учимся программировать». – СПб.: БХВ-Петербург.
7. Столяров А.В. «Программирование на языке ассемблера NASM для ОС Unix» (онлайн-учебник).
8. Портал [Habr.com](https://habr.com)
9. GeeksforGeeks. (2022). Introduction of Assembler. Retrieved March 22, 2023, from <https://www.geeksforgeeks.org/introduction-of-assembler/>
10. GNU Project. (2023). Using as: The GNU Assembler. Binutils Documentation 2.40. Retrieved from <https://sourceware.org/binutils/docs-2.40/as/>
11. IBM. (2022). Assembler language reference. IBM Documentation for AIX 7.3. Retrieved from <https://www.ibm.com/docs/en/aix/7.3?topic=programming-assembler-language-reference>
12. Intel Corporation. (2021). *Intel 64 and IA-32 Architectures Software Developer Manuals*. Retrieved from <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>
13. Microsoft. (2023). Microsoft Macro Assembler reference. Microsoft Docs. Retrieved from <https://learn.microsoft.com/en-us/cpp/assembler/masm/microsoft-macro-assembler-reference>
14. NASM Development Team. (2020). The Netwide Assembler: NASM. Официальная документация версии 2.15.05. Retrieved from <https://nasm.us/doc/>

15. OSDev Wiki Contributors. (2023). Linkers. OSDev Wiki. Retrieved March 22, 2023, from <https://wiki.osdev.org/Linkers>
16. R. (2022). Assembly - Quick Guide. TutorialsPoint. Retrieved from https://www.tutorialspoint.com/assembly_programming/assembly_quick_guide.htm
17. University of Virginia. (2023). x86 Assembly Guide. Computer Science Department. Retrieved from <https://www.cs.virginia.edu/~evans/cs216/guides/x86.html>